Serverless Model Deployment & Pipeline Orchestration on AWS

LEARNING OUTCOMES ASSESSED

- LO8 — Develop serverless applications using AWS Lambda for MLOps workflows.
- LO5 — Apply continuous delivery and orchestration to an ML workflow.

Scenario

For the capstone assignment you bring the module together: deploy a model as a serverless function and orchestrate a data-and-ML pipeline around it. If you do not have AWS access, you may use a local emulator (AWS SAM local or LocalStack) — state clearly in your report which you used.

Requirements

Serverless inference

- Package a trained model as an AWS Lambda function whose handler performs inference on an incoming event or request.
- Expose or trigger the function (API Gateway, a function URL, or an event source) and demonstrate a successful invocation.
- Define and deploy the function as code using the AWS CLI, AWS SAM, or the Serverless Framework (Infrastructure as Code).

Orchestration

- Build an Apache Airflow DAG that orchestrates a pipeline: e.g. extract → validate → transform → train/deploy.
- Embed a data-quality gate (Great Expectations or equivalent) as a task between stages.
- Follow DAG best practice: idempotent, atomic tasks with no secrets hard-coded.

Analysis

- Identify the AWS services you used (or emulated) and map them to their MLOps roles.
- Discuss the trade-offs of serverless (cost, cold starts, scaling) and one relevant future trend in MLOps.

Deliverables

- A repository containing the Lambda handler, the IaC/deployment definition, and the Airflow DAG.
- Evidence of a successful function invocation and of the DAG running (screenshots and/or logs).
- A written report (see below).

What Your Report Must Contain

- A short introduction: the problem, your dataset and your overall approach.
- An architecture or workflow diagram of your solution.
- Your serverless design: the handler, how it is triggered, and how it is deployed as code.

- Your Airflow DAG: its tasks, dependencies and the data-quality gate.
- Your service-to-role mapping and your serverless trade-off analysis.
- Results with evidence (screenshots, logs, tables) and a brief discussion of what they show.
- A reflection on limitations and what you would improve next.

Marking Scheme

This assignment is marked out of 15 using the module's programming-assignment rubric:

Criterion	What is assessed	Marks
Analysis & Code Quality	Sound approach; correct, clean, well-structured and readable code	5
Application of Concepts	Correct, deliberate use of the MLOps concepts from the relevant units	5
Presentation & Elaboration	Clear report; diagrams; decisions explained and justified	3
Timely Submission & Language	Submitted on time; clear, correct written English	2
Total		**15**

Submission & Integrity

- Submit all code to the version-control system allocated by the HoD, SWE, and include the repository link in your report.
- Submit the report in the format prescribed by the module tutor, by the deadline. Late work is penalised under the Timely Submission criterion.
- Your repository must contain a clear README with setup and run instructions, plus a requirements file so your work is reproducible.
- Work must be your own. Cite any sources, libraries or code you build upon. Where AI tools are used, follow the module and institutional policy and declare their use.

Further Reading

- Gift, N. & Deza, A. (2021). Practical MLOps: Operationalizing Machine Learning Models. O'Reilly.
- Pote, S. (2023). Machine Learning in Production: Master the art of delivering robust ML solutions with MLOps. BPB.
- Ping, D. (2023). The Machine Learning Solutions Architect Handbook. Packt.
- AWS Lambda Developer Guide — https://docs.aws.amazon.com/lambda
- Apache Airflow documentation — https://airflow.apache.org/docs